

Практическое задание: Сервис уведомлений

1. Постановка задачи

Необходимо спроектировать сервис уведомлений, который позволяет другим системам отправлять пользователям сообщения по различным каналам доставки.

Система должна поддерживать базовый сценарий:

- внешний сервис формирует запрос на отправку уведомления;
- система принимает запрос и определяет канал доставки;
- система подготавливает сообщение;
- система отправляет уведомление пользователю;
- система сохраняет результат доставки.

Сервис должен поддерживать как минимум следующие каналы:

- email;
- push-уведомления;
- SMS.

Цель проекта — предложить архитектуру highload-системы, способной обрабатывать большой поток уведомлений, поддерживать несколько каналов доставки, корректно работать с временно недоступными внешними провайдерами и не допускать неконтролируемого дублирования сообщений.

2. Минимальный глоссарий

Уведомление — сообщение, которое система должна доставить пользователю.

Канал доставки — способ отправки уведомления: email, push, SMS.

Шаблон — заготовка сообщения с переменными параметрами.

Провайдер — внешний сервис, через который выполняется доставка уведомления.

Retry — повторная попытка отправки уведомления после ошибки.

Rate limiting — ограничение интенсивности отправки.

Deduplication — предотвращение повторной отправки одного и того же уведомления.

Статус доставки — текущее состояние уведомления.

3. Функциональные требования

3.1. Приём запросов на отправку

Система должна позволять внешним сервисам:

- отправлять запрос на создание уведомления;
- указывать получателя;
- указывать канал доставки;
- передавать данные для формирования текста сообщения.

Каждое уведомление должно содержать как минимум:

- `notification_id`;
- `recipient_id` или адрес получателя;
- канал доставки;
- тип уведомления;
- время создания;
- текущий статус.

3.2. Поддержка каналов доставки

Система должна поддерживать как минимум:

- отправку email;
- отправку push-уведомлений;
- отправку SMS.

Для каждого канала допускаются собственные ограничения на формат и размер сообщения.

3.3. Работа с шаблонами

Система должна:

- использовать шаблоны сообщений;
- подставлять в шаблон переданные параметры;
- формировать итоговое сообщение перед отправкой.

3.4. Отправка уведомлений

Система должна:

- отправлять уведомление через соответствующий канал;

- взаимодействовать с внешним провайдером доставки;
- фиксировать результат отправки.

3.5. Повторные попытки отправки

Система должна:

- выполнять повторную попытку отправки при временной ошибке;
- ограничивать число повторных попыток;
- переводить уведомление в финальное неуспешное состояние после исчерпания лимита `retry`.

3.6. Защита от дублей

Система должна предотвращать повторную отправку одного и того же уведомления при:

- повторной отправке запроса клиентом;
- повторной обработке внутреннего сообщения;
- временных сбоях в отдельных компонентах системы.

3.7. Статусы уведомления

Система должна поддерживать как минимум следующие статусы:

- `created`;
- `processing`;
- `sent`;
- `failed`.

3.8. История уведомлений

Система должна позволять:

- получать историю отправленных уведомлений;
- видеть канал доставки;
- видеть время отправки;
- видеть текущий статус доставки.

4. Нефункциональные требования

4.1. Нагрузка

Система должна выдерживать:

- до **20 000 запросов на отправку в секунду** в пике;
- неравномерную нагрузку с кратковременными всплесками.

Основной характер нагрузки:

- высокая интенсивность записи;
- асинхронная обработка отправки;
- зависимость от внешних провайдеров доставки.

4.2. Производительность

Требования к производительности:

- приём запроса на отправку — P95 не более **100 мс**;
- постановка уведомления в обработку должна происходить практически сразу;
- итоговая доставка может быть асинхронной и зависеть от канала и внешнего провайдера.

4.3. Надёжность

Система должна обеспечивать:

- отсутствие потери принятых уведомлений;
- устойчивость к сбоям отдельных экземпляров сервисов;
- корректную обработку повторных запросов;
- корректную работу при временной недоступности провайдера.

4.4. Консистентность

Для самого факта приёма уведомления важна надёжная запись.

Для истории и вторичных представлений допускается eventual consistency.

Система не должна бесконтрольно отправлять дубли одного и того же уведомления.

4.5. Масштабируемость

Система должна горизонтально масштабироваться по следующим контурам:

- приём запросов;
- обработка и маршрутизация уведомлений;
- отправка по различным каналам;
- хранение истории уведомлений.

4.6. Ограничения и контроль нагрузки

Система должна предусматривать:

- ограничение скорости отправки;

- защиту от перегрузки внешних провайдеров;
 - возможность изолировать проблемы одного канала доставки от остальных.
-

5. Что требуется спроектировать

В рамках задания необходимо подготовить техническое решение, содержащее:

1. Краткое описание системы и её назначения.
 2. Основные пользовательские и технические сценарии.
 3. Модель уведомления и его жизненного цикла.
 4. Архитектуру приёма, маршрутизации и отправки уведомлений.
 5. Подход к работе с шаблонами.
 6. Подход к getty и обработке ошибок провайдеров.
 7. Подход к предотвращению дублей.
 8. Подход к хранению истории уведомлений.
 9. Подход к масштабированию и обеспечению отказоустойчивости.
 10. Основные компромиссы выбранного решения.
-

6. Ожидаемый результат

По итогам выполнения задания должно быть подготовлено техническое решение в формате system design, включающее:

- описание требований;
- ключевые сценарии;
- архитектурную схему;
- описание основных компонентов;
- модель данных;
- обоснование выбора архитектуры;
- анализ компромиссов и ограничений решения.